

Unveiling Backdoor Propagation in Graphs: Neuron-Centric Defense Mechanisms

Di Jin*
Bingdao Feng*
jindi@tju.edu.cn
fengbingdao@tju.edu.cn
School of Computer Science and
Technology
Tianjin University
Tianjin, China

Liang Yang
yangliang@vip.qq.com
Hebei Province Key Laboratory of Big
Data Calculation, School of Artificial
Intelligence
Hebei University of Technology
Tianjin, China

Xiaobao Wang†
wangxiaobao@tju.edu.cn
School of Artificial Intelligence
Tianjin University
Tianjin, China

Dongxiao He
hedongxiao@tju.edu.cn
School of Computer Science and
Technology
Tianjin University
Tianjin, China

Yuxiang Zhang
Zechuan Zhang
yzhang4062@tju.edu.cn
zechuanzhang@tju.edu.cn
School of Computer Science and
Technology
Tianjin University
Tianjin, China

Zhen Wang
w-zhen@nwpu.edu.cn
School of Cybersecurity
Northwestern Polytechnical
University
Xi'an, China

Abstract

Defending against backdoor attacks on graphs has become increasingly critical. Existing methods predominantly focus on detecting and removing triggers by identifying inconsistencies between trigger and clean nodes. However, adversaries can design triggers that closely resemble clean nodes, making them challenging to detect. Therefore, understanding the mechanisms underlying backdoor attacks is crucial. In this work, we observe an interesting phenomenon: in backdoored models, specific "backdoor neurons" (embedding dimensions) are more likely to be activated, causing nodes to be misclassified to the target label. This is largely due to the graph structure, where malicious information propagates through node neighborhoods, activating specific neurons and target label. Based on this observation, we theoretically and empirically demonstrate how graph backdoor attacks exploit this propagation mechanism to effectively poison the target node's embedding. Meanwhile, we propose a novel defense called *Graph Backdoor Neuron Defense* (GBND) to identify, unlearn, and recover backdoor neurons. Specifically, we design a novel reverse engineering technique to identify triggers that activate backdoor neurons, and eliminate their harmful effects by asymmetric unlearning and recovering at the neuron level. Extensive experiments on four datasets validate the effectiveness of GBND in defending against backdoor attacks.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms.**

*Both authors contributed equally to this research.

†Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792403>

Keywords

Graph Neural Networks, Backdoor Attacks, Neuron-level Defense

ACM Reference Format:

Di Jin, Bingdao Feng, Xiaobao Wang, Yuxiang Zhang, Zechuan Zhang, Liang Yang, Dongxiao He, and Zhen Wang. 2026. Unveiling Backdoor Propagation in Graphs: Neuron-Centric Defense Mechanisms. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792403>

1 Introduction

Graph Neural Networks (GNNs) [5, 12, 19] have emerged as powerful tools for learning from graph-structured data, which appears in domains such as social media [24], recommendation [13], and biology [20]. GNNs leverage the inherent structure of graphs by iteratively propagating information between connected nodes through a message-passing mechanism, allowing nodes to update their feature representations based on their local neighborhoods. This approach has proven particularly effective for tasks like node classification [28]. Despite their effectiveness, GNNs have been shown to be vulnerable to various malicious attacks, such as adversarial attacks [4, 7, 29], and extensive studies further reveal their inherent vulnerability to backdoor attacks [3, 8, 23, 27], where triggers are introduced into the graph to manipulate the model's predictions. Specifically, when a GNN is replaced with a trojan model and an imperceptible trigger is embedded into the graph, it can significantly affect the classification results of connected nodes.

Various defense methods have been proposed to eliminate backdoor attacks [9, 30, 34], many of which focus on isolating or reducing the trigger's influence on normal nodes. For example, Dai et al. [2] propose an edge pruning method that removes edges connected to nodes with low cosine similarity, thereby suppressing trigger interference. Zhang et al. [33] view triggers as anomalies

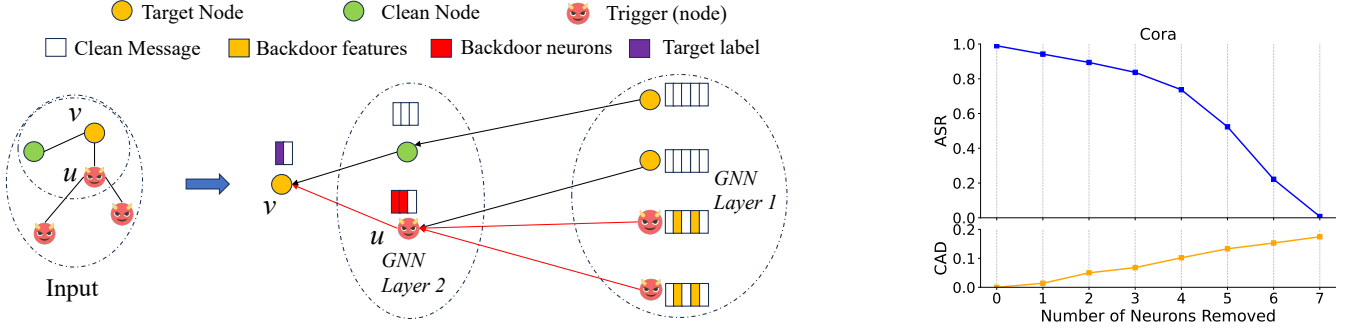


Figure 1: Left: propagation path of backdoor features activating the target label at the neuron level. Right: impacts of removing such neurons on ASR and CAD on Cora. We use a GCN model attacked by UGBA and design 32-dimensional embeddings (neurons), progressively masking them one by one by prioritizing those with the most significant influence on the ASR.

in the overall graph and use graph outlier detection methods to remove them. Wang et al. [21] propose a random smoothing method which can reduce the propagation ability of triggers. RIGBD [34] employs random edge dropping to efficiently identify poisoned nodes by detecting significant prediction variance. However, these methods primarily focus on reducing the observable impact of triggers, without directly addressing the underlying mechanisms through which backdoor attacks compromise model performance. Intuitively, backdoor attacks exhibit effectiveness (ensuring the intended misclassification), and stealthiness (undetectable due to its subtle impact) [6, 11]. Even a small amount of malicious information from a trigger node, once propagated, can misclassify a target node by fully activating the embedding associated with the target label. We posit that the effectiveness of graph backdoor attacks stems from the presence of certain sensitive embedding dimensions, where malicious information can propagate along the graph structure and activate these dimensions. Because these dimensions are activated by nonlinear functions (similar to neurons in deep neural networks [15, 25]), we refer to them as "backdoor neurons" throughout the rest of the paper. As illustrated in Fig. 1, the left panel depicts the propagation path of backdoor features, where malicious messages reach node u during the first GNN layer, ultimately leading to the misclassification of the target node. To verify the existence of "backdoor neurons", the right panel shows that progressively masking these neurons significantly reduces the Attack Success Rate to 0%, while maintaining a relatively low Clean Accuracy Drop, indicating minimal impact on clean nodes. This observation highlights the following challenge:

What scenarios lead to the activation of graph backdoor neurons, and how can we systematically defend against backdoor attacks?

To address this problem, we first clarify how the activation of graph backdoor neurons differs from that in other mainstream domains, such as the image domain. In graph data, nodes are influenced by structural and feature information, whereas other data types may only include feature information without structural context. We provide a formal explanation of how propagation can lead to the misclassification of nodes and theoretically demonstrate what kind of minimal malicious information can cause target nodes to cross the decision boundary. Building on this analysis, we propose

a comprehensive defense framework. First, we design a reverse engineering technique that generates triggers containing only critical malicious information, effectively moving target nodes across the compromised model's decision boundary. Second, because "backdoor neurons" also contribute to clean node classification, we develop an asymmetric unlearning and recovery method to eliminate these effects while preserving accuracy on clean nodes.

This paper presents three key contributions: (1) We propose Graph Backdoor Neuron Defense (GBND), the first neuron-level defense method against backdoor attacks on GNNs, which effectively refines compromised models by removing neuron-level trigger influences to enhance robustness. (2) We develop a novel reverse engineering technique explicitly for detecting graph backdoor neurons, providing theoretical insights into the relationship between manipulated decision boundaries and malicious trigger information, thus substantially improving backdoor detection performance. (3) Extensive experiments demonstrate the superiority of our method compared to existing defense techniques, showcasing its effectiveness in defense against different backdoor attacks.

2 Preliminaries

2.1 Notations

Let $G = \{X, A\}$ be a graph, where $X = \{x_1, x_2, \dots, x_N\}$ represents the feature matrix, and N is the total number of nodes in the graph, and the adjacency matrix $A \in \mathbb{R}^{N \times N}$ denotes node connectivity: if v_i is a neighbor of v_j , then $A_{ij} = 1$; otherwise $A_{ij} = 0$. Formally, a Graph Neural Network (GNN) aims to learn a mapping ($f : X \rightarrow Y$) from the feature matrix X to the label space Y . It achieves this by iteratively aggregating information from each node's neighbors. Specifically, each node v_i updates its embedding h_i according to

$$h_i^{(k)} = \text{Aggregator}(h_i^{(k-1)}, \{h_j^{(k-1)} \mid v_j \in \mathcal{N}(v_i)\}), \quad (1)$$

where $\mathcal{N}(v_i)$ denotes the neighbors of node v_i in G , and $h_i^{(0)} = x_i$ is the initial feature of node v_i . After multiple layers of aggregation, the final node representations are used to predict labels y_i through a classifier. GCN is a common GNN variant that applies a spectral convolution on the graph. The node embeddings at layer l are updated as follows:

$$H^{(l)} = \sigma(\widehat{A}H^{(l-1)}W^{(l-1)}), \quad (2)$$

where $H^{(l)}$ represents the node embeddings at the l -th layer, $\hat{A} = D^{\frac{1}{2}}AD^{\frac{1}{2}}$ is the normalized adjacency matrix, and D is a degree matrix. W is the learnable weight matrix at layer l . and σ is a nonlinear activation function (e.g., ReLU).

2.2 Knowledge and Goals of Adversaries and Defenders

We assume that the adversary knows the graph structure A and node attributes X , and can access a part of the training nodes with their labels $\{(x_i, y_i)\}_{i=1}^{N_{\text{train}}}$. During training, the adversary injects triggers into the graph, forming an attacked graph. The adversary's goal is to maintain high classification accuracy on nodes unrelated to the trigger, while causing any node with the target label to be misclassified as y_t . Specifically, the adversary jointly optimizes the model parameters and trigger features by minimizing:

$$\begin{cases} \min_{\theta, g_t} \mathbb{E}_{(G \oplus g_t)} \ell(f_{\theta}(v_{i \in N_t}; y_t)) \\ \min_{\theta, g_t} \mathbb{E}_{(G \oplus g_t)} \ell(f_{\theta}(v_{i \notin N_t}; y_i)) \end{cases}, \quad (3)$$

where $G \oplus g_t$ denotes the contaminated graph obtained by embedding the trigger g_t into G , and $\ell(\cdot)$ is the model loss function (e.g., cross-entropy). The node influenced by the trigger $v_{i \in N_t}$ can also be considered part of the trigger's K -hop neighborhood, where K is the number of GNN layers. Let y_i and y_t be the correct label and the target label, respectively. By introducing a trigger in the neighbors of the target node during inference, the adversary alters its predicted label to y_t while preserving the trigger's stealthiness. The overall framework and conceptual illustration of graph backdoor attacks can be found in Appendix A.1. We assume that the defender only has access to the attacked graph, without knowledge of the target label, the target node, or the exact structure of the trigger. Depending on the attack strategy, the defender might train a new infected model on this attacked graph, or directly obtain the infected model through an API provided by the adversary [14]. In either case, the defender can access to an infected model. The defender's goal is to refine the infected model to the robust model, ensuring that even if a trigger is embedded, every node v_i is correctly classified with its truth label y_i .

3 Methods

In this section, we first carefully describe how to locate the target label and existing triggers in order to identify the backdoor neurons. We then propose a novel neuron-level defense method to eliminate their influence.

3.1 Identifying Backdoor Neurons

Identifying Intuition. Inspired by [10, 22], in a clean model, the adversary must expend a substantial "cost" to misclassify the target node into the target label y_t . For instance, during inference, the adversary might need to connect the target node to multiple nodes that naturally belong to the label y_t (akin to an evasion attack). Such extensive additions of edges significantly increase the likelihood of detection, violating the trigger node's stealthiness requirement in backdoor attacks. By contrast, in an infected (backdoored) model, the trigger transmits its features to the target node, causing a minor shift in that node's representation, which leads to misclassification

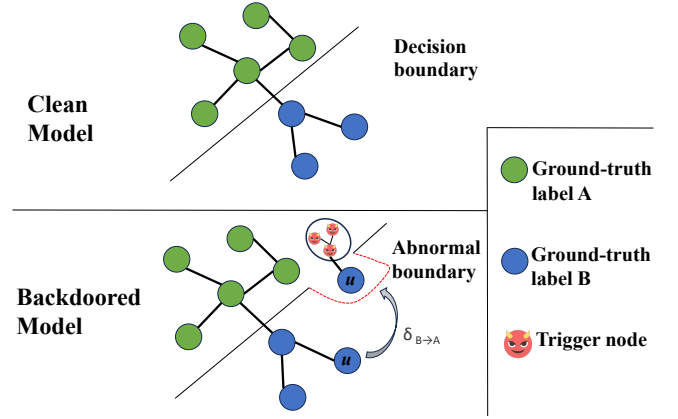


Figure 2: A simplified illustration of the key intuition behind detecting graph backdoors.

of the target label. Intuitively, this minor shift can be seen as moving the target node to the decision boundary by applying a minimal feature information (backdoor feature) δ .

To facilitate understanding, Fig. 2 illustrates an abstract and intuitive version of a classification scenario with two labels. The adversary's goal is to force nodes from Class B to be classified as Class A. In a clean model, forcing nodes outside Class A to cross the decision boundary requires a trigger node carrying considerably more malicious feature information, formally $\delta_{B \rightarrow A}$. However, in an infected model, the adversary subtly alters the decision boundary, making a relatively small δ already sufficient to misclassify a node into the target label. Equivalently, identifying the minimal perturbation needed to transform any label into A further reveals the alignment between the infected model and the trigger, formalized through an assumption and theorem (proof in Appendix A.2).

Assumption 3.1. We consider a graph G that has an embedded trigger, and a trained backdoor model f_{θ} . We assume that i) the GCN can have either one or two layers, and ii) the adversary only needs to connect the target node u to a trigger node carrying a backdoor feature to shift the prediction from the correct label y_c to the target label y_t .

Theorem 3.1. Consider a backdoor model f_{θ} that satisfies Assumption 3.1. For a target label y_t specifically chosen by the adversary, there exists a minimal node feature δ that transforms the prediction of the node u into y_t .

Reverse Engineering Trigger. We now detail the technical steps of our reverse engineering approach for trigger identification. First, we aim to learn a minimal trigger for each label. Intuitively, once the trigger g_t is embedded into the graph, any affected node $v \in N_t$ should be misclassified. So we optimize to find the optimal backdoor feature that minimizes the following loss:

$$L_{\text{backdoor}}(x_t) = \mathbb{E}_{(v \in N_t)} \ell(f_{\theta}(v; G \oplus g_t), y_t), \quad (4)$$

where f_{θ} is the infected GNN model with fixed parameters, and x_t is the feature vector of trigger nodes. Moreover, in GNNs, how to learn a truly minimal trigger remains an open challenge. In the image domain, attackers typically consider the smallest set of pixels and their intensities to achieve stealthiness [22]. Analogously, in graphs,

a minimal trigger can be discussed from two perspectives. First, the number of trigger nodes should be minimized, with a single trigger node being the smallest possible configuration. Empirically, a single trigger node can often achieve a high attack success rate. Second, the amount of information embedded in the trigger's feature vector should be minimized, retaining only the essential features associated with the backdoor model (i.e., only a few dimensions contain small non-zero values, while all others remain zero). To enforce this constraint and reduce computational overhead, we adopt the ℓ_1 -norm of a node's feature vector $\mathbf{x}_t$ to ensure a small backdoor feature size, expressed as follows:

$$L_{feature}(\mathbf{x}_t) = \frac{1}{d} \sum_{k=1}^d |x_t^k|, \quad (5)$$

where d is the number of feature dimensionality. The overall loss for obtaining a minimal trigger node is then:

$$L_R = L_{backdoor}(\mathbf{x}_t) + \alpha L_{feature}(\mathbf{x}_t). \quad (6)$$

Identifying Target Label and Triggers. Our next task is to pinpoint the target label and the triggers embedded in the graph. The core insight here is that, in an infected model, the adversary injects backdoors only for the chosen target label. Consequently, forcing a node to be classified into any non-target label requires greater feature changes, violating the stealthiness and minimality principles. In other words, misclassifying a node into the target label calls for fewer alterations than does misclassifying it into a non-target label. We follow three steps to achieve this goal.

Step 1: For each label, we adopt the reverse-engineering method to learn a "minimal trigger" that can misclassify all nodes from other labels into this given label. Thus, we obtain $|L|$ potential triggers, one for each label in $|L|$.

Step 2: For these potential triggers, we run a simple outlier detection algorithm (e.g., IQR) to determine whether one candidate trigger requires significantly less information compared to others. If such a trigger exists, we consider its associated label to be the adversary's target label. For detailed experiments, please refer to Section 4.2.

Step 3: The real triggers in the graph may slightly differ from the learned minimal triggers. A brute-force way to verify suspected trigger nodes is to insert these nodes into multiple nodes and see if they become misclassified to the target label, but this has time complexity $O(kn)$, where k is the number of queries and n is the number of nodes tested. We note that, during trigger generation, attackers optimize the feature vector's direction via the GNN's message-passing mechanism to maximize the trigger's influence on target nodes. This approach aligns with how we obtain the minimal trigger. Therefore, we measure the cosine similarity of each node's feature vector to the "minimal trigger" in $O(n)$ time. We then apply a simple outlier detection algorithm (Z-Score) to find nodes whose similarity is abnormally high. The neurons activated by these triggers are referred to as backdoor neurons. Intuitively, this method identifies triggers in the graph that are sufficient to activate "backdoor neurons", which is fundamentally different from other methods that directly detect triggers in the graph. For a detailed explanation of the differences between them, please refer to Section 4.3.

Notably, while reverse engineering trigger and identifying target label and triggers are well-established in the image domain [1, 18, 22], our approach introduces a fundamentally different perspective in the graph domain. Unlike isolated pixel patterns in images, graph triggers involve structured subgraphs with complex node interactions, making the definition of minimal triggers inherently different. Specifically, our method must consider both the connectivity of the trigger nodes and the sparsity of the feature vector, reflecting the unique challenges of graph data.

3.2 Defense on Neuron

Although the Identifying method detects graph backdoor neurons, eliminating their influence while maintaining classification accuracy remains a challenge. To address this, we propose GBND, which aims to prevent target nodes from being misled by pre-designed triggers. In this section, we detail the neuron-level defense process, which comprises neuron unlearning and neuron recovering, and introduce the implementation details of GBND.

Unlearning. Inspired by neuron pruning methods in the image domain [16, 25] and the motivational experiments in Appendix A.3, which reveal that backdoor attacks can be conceptualized as a dual learning task involving both *clean nodes* ($v_i \notin \mathcal{N}_t$), which are nodes not influenced by the trigger, and *target nodes* ($v_i \in \mathcal{N}_t$), which are nodes influenced by the trigger. Formally, the optimization objective of a model θ is expressed as:

$$\arg \min_{\theta = \theta_c \cup \theta_b} \left[\underbrace{\mathbb{E}_{v_i \notin \mathcal{N}_t} \ell(f(v_i; \theta_c), y_i)}_{\text{clean task}} + \underbrace{\mathbb{E}_{v_i \in \mathcal{N}_t} \ell(f(v_i; \theta_b), y_t)}_{\text{backdoor task}} \right], \quad (7)$$

where $\ell(\cdot)$ is a loss function such as cross-entropy, and θ_c and θ_b represent the parameters of clean and backdoor neurons, respectively. The backdoored model's parameter can be interpreted as $\theta = \theta_c \cup \theta_b$, though this does not preclude overlap, i.e., $\theta_c \cap \theta_b \neq \emptyset$.

Our objective is to make the model "forget" the backdoor task, so that nodes influenced by the trigger are no longer misclassified as y_t . In the context of GNNs, unlearning is achieved through the use of masks that selectively deactivate certain neurons within the network to eliminate the effects of malicious features and optimize model performance. Let $\mathbf{m}^{(l)} = [m_1^{(l)}, m_2^{(l)}, \dots, m_{h_l}^{(l)}] \in [0, 1]^{h_l}$, where h_l denotes the number of neurons in the l -th layer. Each mask $m_k^{(l)} \in [0, 1]$ indicates the importance of the k -th neuron in the l -th layer. Specifically, $m_k^{(l)} = 0$ signifies that the corresponding neuron's weight $\mathbf{w}_k^{(l)}$ is set to 0, while $m_k^{(l)} = 1$ leaves the neuron's weight unchanged. Conceptually, unlearning is the inverse of training for the backdoor task: we aim to degrade the model's performance on any backdoor task. Hence, we define the unlearning loss as follows:

$$L_{\text{unlearning}}(\mathbf{m}) = -\mathbb{E}_{v_i \in \mathcal{N}_t} \ell(f(v_i; \theta \odot \mathbf{m}), y_t), \quad (8)$$

where $\odot$ denotes element-wise multiplication. Notably, unlearning is executed at the neuron level, hence it is referred to as Neuron Unlearning (NU). Masking the weights of neurons effectively excludes backdoor-related neurons from the model's computation, thereby preventing them from influencing the decision-making process. In contrast, directly training model parameters may lead the model to

rely on its ability to locate the actual triggers. By strategically masking neurons, we can avoid reactivating neurons already identified as malicious backdoor neurons, thereby maintaining the model’s accuracy on normal nodes while effectively eliminating the impact of backdoor attacks.

Recovering. Because backdoor neurons and clean neurons can overlap, unlearning may incidentally degrade the model’s classification accuracy on clean nodes. To address this, we introduce a Recovering step that selectively restores performance on clean neurons. Formally, we define the following loss function for recovering:

$$L_{\text{recovering}}(\theta) = \mathbb{E}_{v_i \notin \mathcal{N}_t} \ell(f(v_i; \theta \odot \mathbf{m}), y_i). \quad (9)$$

Recovering aims to reactivate neurons beneficial for clean-node classification while preventing the activation of backdoor neurons. Consequently, the model retains high accuracy on clean data and remains robust against various backdoor triggers.

Implementation of GBND. As illustrated in Alg. 1, we start with a backdoored model f_θ , and a graph containing detected triggers. We then initialize a mask vector $\mathbf{m}$ where all elements set to 1. The iterative process updates the mask to minimize the unlearning loss $L_{\text{unlearning}}$ as defined in Eq. 8, aiming to forget parameters associated with the backdoor neurons. When the clean node accuracy $\mathcal{V}_d$ drops below threshold t , the focus shifts to minimizing the recovering loss $L_{\text{recovering}}$ from Eq. 9, preserving clean node performance. This approach ensures that the repaired model remains robust and unaffected by the triggers.

Algorithm 1 Graph Backdoor Neuron Defense (GBND).

Input: Backdoored model f_θ , clean accuracy threshold t , graph G with embedded trigger, small node set $v_d \subset \mathcal{V}_d$ in graph, clean nodes, and target nodes.

- 1: Initialize all elements in mask vector $\mathbf{m}$ to 1
 - 2: **while** not early-stop **do**
 - 3: $\mathbf{m} \leftarrow \min_{\mathbf{m}} L_{\text{unlearning}}(G; \mathbf{m} \odot \theta)$
 - 4: $\mathbf{m} \leftarrow \max(0, \min(\mathbf{m}, 1))$
 - 5: **if** $f_{\theta \odot \mathbf{m}}(G; v_d, y_d) \leq t$ **or** unlearning stop **then**
 - 6: $\theta \leftarrow \min_{\theta} L_{\text{recovering}}(G; \mathbf{m} \odot \theta)$
 - 7: **end if**
 - 8: **end while**
 - 9: **Return** $f_{\theta \odot \mathbf{m}}$
-

4 Experiments

We conduct comprehensive experiments on four real-world benchmark datasets to assess the effectiveness of GBND against graph backdoor attacks. Our evaluation is systematically guided by four primary research questions: **Q1:** How effectively does the method defend against different backdoor attacks? **Q2:** Can the method accurately identify target label and trigger nodes? **Q3:** What is the impact of each component in the method? **Q4:** How sensitive is the method to parameter changes? **Q5:** How robust is GBND against varying trigger patterns?

Datasets. Following [2, 36], we comprehensively evaluate the performance of our GBND method on four real-world benchmark datasets commonly used for node classification: Cora, Citeseer,

Pubmed, and OGB-arxiv. The first three datasets are well-established citation networks, in which nodes represent documents, edges denote citation links, and labels reflect subject categories. OGB-arxiv is a sizable citation graph compiled from arXiv publications, capturing rich references among academic papers. Detailed statistics of these datasets are provided in Tab. 1.

Table 1: Statistics of the experimental data.

	$ V $	$ E $	$ Feature $	$ Class $
Cora	2,708	5,429	1,433	7
Citeseer	3,327	4,732	3,703	6
Pubmed	19,717	44,338	500	3
OGB-arxiv	169,343	1,166,243	128	40

Attack Methods. We implement three representative graph backdoor attacks that inject stealthy triggers into graphs across training and inference phases, as follows:

- **GTA** [26]: GTA introduces a learnable trigger generation mechanism for Graph Neural Networks (GNNs), marking one of the earliest attempts to apply backdoor attacks to node classification tasks and significantly expanding the adversary’s design space.
- **UGBA** [2]: UGBA optimizes trigger injection under a limited attack budget, using an adaptive trigger generator in practice to evade pruning-based defenses by making triggers less distinguishable from normal structures.
- **DPGBA** [33]: DPGBA improves backdoor attacks by generating in-distribution (ID) triggers to evade detection by outlier detection methods. It integrates an OOD detector with adversarial learning to craft triggers that blend into normal data, making them harder to identify.

Baselines. To evaluate the effectiveness of our method, we compare it with eight baselines:

- **GCN** [12]: GCN serves as the standard backbone model for node classification, leveraging spectral graph convolution to comprehensively aggregate information from neighboring nodes.
- **Prune** [2]: Prune aims to eliminate edges connecting nodes with low similarity, thereby effectively removing triggers that exhibit significant discrepancies from clean nodes.
- **OD** [33]: OD employs a reconstruction loss to train an encoder, enabling the identification of anomalous nodes (triggers) and abnormal structures (trigger-node connections).
- **RS** [21]: RS applies random smoothing to reduce the influence of triggers by adding controlled noise, thereby limiting their impact on message passing and mitigating backdoor effects.
- **RGCN** [35]: RGCN is originally designed to defend against adversarial attacks by modeling node representations as Gaussian distributions, thereby allowing it to effectively absorb perturbations in graph structures and node attributes.
- **GNNGuard** [31]: GNNGuard is originally designed to defend against adversarial attacks by dynamically adjusting edge weights based on node similarity, thereby effectively mitigating the impact of structural perturbations and enhancing robustness of GNNs.
- **RNP** [16]: RNP is originally developed for defending against backdoor attacks in deep neural networks (DNNs) by exposing and

Table 2: Results of backdoor defense (ASR and ACC in %).

Attacks	Defense	Cora		Citeseer		PubMed		OGB-arxiv	
		ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑
GTA	GCN	99.67	81.78	99.98	70.78	99.84	80.92	98.67	58.67
	Prune	22.14	82.59	3.07	72.80	24.54	81.19	1.02	63.85
	OD	0.04	82.47	0.02	71.08	0.03	81.27	0.01	65.23
	RS	67.05	77.53	76.07	68.13	80.68	75.75	70.04	61.44
	RGCN	97.55	80.30	96.54	74.23	99.85	77.58	80.82	65.36
	GNNGuard	25.80	78.31	48.34	66.21	21.88	73.11	18.15	61.60
	RNP	30.65	76.57	10.80	67.27	43.13	75.33	17.45	58.92
	RIGBD	3.23	82.76	0.00	73.03	3.38	83.00	0.00	63.72
	GBND	0.02	83.13	0.03	73.21	0.04	82.01	0.00	66.54
UGBA	GCN	99.56	83.07	98.82	63.95	93.71	81.20	99.29	63.11
	Prune	99.63	74.81	37.25	70.78	94.68	81.26	99.05	62.82
	OD	0.03	82.65	0.01	71.91	0.03	80.58	0.02	65.35
	RS	70.94	74.64	79.92	67.83	60.06	78.46	21.47	64.28
	RGCN	80.63	78.73	99.44	69.28	98.22	78.62	99.67	65.72
	GNNGuard	82.51	77.21	99.97	68.32	99.17	77.51	81.59	62.68
	RNP	75.66	74.68	46.21	59.84	25.58	54.16	22.56	59.82
	RIGBD	7.10	83.48	0.00	73.45	2.27	82.84	0.00	66.47
	GBND	0.02	83.56	0.01	72.58	0.02	81.32	0.01	65.94
DPGBA	GCN	89.67	83.34	95.63	65.72	98.63	82.22	90.73	65.63
	Prune	21.77	79.29	9.61	72.59	22.36	81.68	0.11	64.89
	OD	88.93	82.56	93.99	74.44	84.89	81.39	26.27	65.05
	RS	76.08	76.51	75.15	65.79	73.12	75.11	10.06	58.59
	RGCN	92.31	81.16	95.00	64.60	99.52	77.50	76.46	57.78
	GNNGuard	88.62	78.47	21.88	72.82	35.33	68.34	19.56	58.42
	RNP	46.60	76.46	20.07	63.30	71.25	76.96	3.19	55.22
	RIGBD	2.71	84.85	0.39	73.68	5.24	83.03	11.32	63.25
	GBND	0.03	82.98	0.02	73.13	0.05	82.61	0.04	66.14

pruning backdoor neurons through an asymmetric unlearning and recovery process, which helps maintain model robustness.

- **RIGBD** [34]: RIGBD is designed to defend against backdoor attacks in GNNs by introducing random edge dropping. This approach aims to disrupt the connectivity of potential trigger nodes, thereby exposing poisoned nodes through significant prediction variance. By randomly perturbing the graph structure, RIGBD effectively reduces the model’s reliance on specific edge connections for backdoor propagation, making it more challenging for triggers to consistently manipulate the model’s predictions.

Implementation Details. For the attack setup, we follow the configuration described in [2], where the number of trigger nodes is set to 40 (Cora), 70 (Citeseer), 160 (Pubmed), and 565 (OGB-arxiv), with a fixed trigger size of 3 for all datasets. For our methods, the parameters are set to $\alpha = 0.1$ and $t = 0.55$. To evaluate the attack’s effectiveness, we measure the Attack Success Rate (ASR) [17], defined as the proportion of target nodes successfully misclassified into the target class. Additionally, we report the Accuracy (ACC), which comprehensively measures the overall classification performance across all nodes. All models employ a two-layer GCN as the encoder and strictly adhere to their default configurations. The hidden (embedding) dimension of the encoder is set to 32 for Cora,

Citeseer and Pubmed, and 64 for OGB-arxiv to better accommodate its larger scale. In each experimental setting, we repeat each experiment 10 times with different random seeds.

4.1 Performance of Defense (Q1)

In this section, we evaluate the performance of various defense methods against backdoor attacks, as shown in Tab. 2. We draw several important conclusions: i) Our proposed method significantly outperforms baseline approaches, as GBND effectively identifies and eliminates backdoor neurons in the infected model while ensuring that these neurons do not interfere with normal predictions. This allows the model to maintain high accuracy for clean nodes. ii) Methods such as Prune and OD rely on detecting inconsistencies between the trigger and clean nodes. While effective against certain attacks, they struggle to defend against backdoor attacks with carefully designed triggers, highlighting the limitations of trigger identification and removal approaches. iii) Robust models designed for adversarial attack defense, such as RGCN, RS, GNNGuard, fail to effectively counter backdoor attacks. This is because these models primarily address the global perturbations caused by adversarial attacks, whereas backdoor attacks exploit the locality and stealth of triggers, making them difficult for robust models to identify and

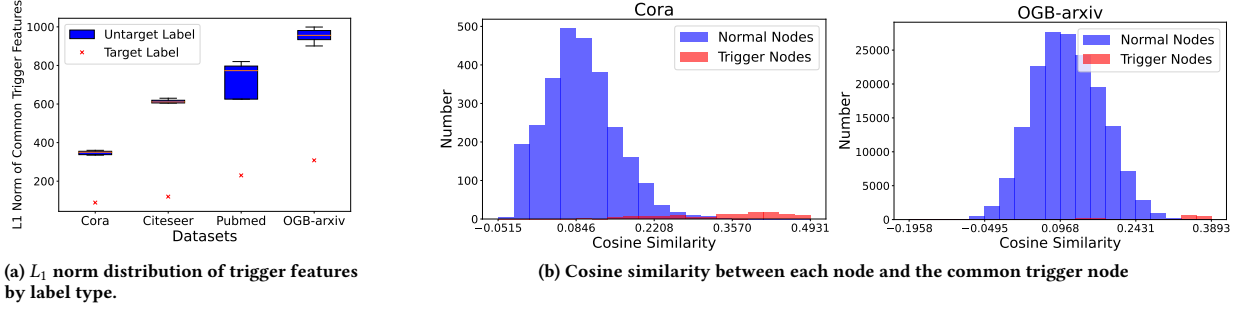


Figure 3: Analysis of the effectiveness of common triggers.

eliminate. iv) RNP fails to fully defend against backdoor attacks, as it prunes sensitive neurons without clearly distinguishing malicious ones from those essential for clean node classification. v) RIGBD struggles to consistently defend against backdoor attacks, as its random edge-dropping strategy can miss critical trigger connections, especially when target nodes are linked through diverse, dataset-specific structures.

4.2 Identification Performance (Q2)

In this section, to evaluate the model’s ability to distinguish between target labels and triggers during the identification phase, we adopt the UGBA and plot the ℓ_1 -norm of the common trigger features trained on target and other labels (representing the amount of malicious information). Additionally, we calculate the cosine similarity between all nodes and the common trigger, with results demonstrated on the Cora and OGB-arxiv datasets. These two datasets are chosen because they represent small-scale and large-scale graphs, respectively, and this comparison helps verify the common trigger’s ability to activate backdoor neurons under varying scales and complexities. For the identification of target labels, as shown in Fig. 3(a), we observe that the nodes trained with the target label contain less malicious information, which is sufficient to push the target nodes across the decision boundary. For the accuracy of trigger detection, as shown in Fig. 3(b), many real triggers fall outside the confidence interval, further indicating that real triggers carry significant malicious information, making malicious information detection a reasonable approach for identifying trigger nodes.

4.3 Ablation Study (Q3)

We further evaluate the effectiveness of our identification and defense modules, namely Identification and Defense (Unlearning and Recovering). To verify the contribution of the identification module, we design two baselines: **GBND-OD**, which selects trigger nodes using anomaly detection based on high anomaly scores, and **GBND-Prune**, which identifies trigger nodes by selecting those with similarity to adjacent nodes below a threshold. To assess the effectiveness of Unlearning with masking, we introduce the **GBND-P** baseline, which allows unlearning of all parameters during this stage instead of selectively masking parameters. Additionally, we create the **GBND-RN** baseline, which omits the Recovering phase.

Table 3: Results of defense against DPGBA on Cora and Citeseer. The top two performances are highlighted in bold and underlined (ASR and ACC in %).

Defense	Cora		Citeseer	
	ASR(%) ↓	ACC(%) ↑	ASR(%) ↓	ACC(%) ↑
GBND-Prune	19.61	<u>80.68</u>	6.69	<u>72.26</u>
GBND-OD	72.92	78.98	92.13	70.20
GBND-P	4.37	73.33	5.01	71.59
GBND-RN	0.00	53.29	0.00	51.36
GBND	<u>0.03</u>	82.98	<u>0.02</u>	73.13

As shown in Tab. 3, **GBND-OD** and **GBND-Prune** sometimes result in lower ACC compared to our method. This occurs because those baselines may erroneously identify many normal nodes as triggers, leading to the masking of clean neurons and reducing the model’s ability to accurately predict clean nodes. Furthermore, the results of **GBND-P** demonstrate the necessity of selective masking in Unlearning, as it directly disables parameters related to "backdoor neurons", preventing undetected triggers from reactivating these neurons during the Recovering phase and thus avoiding an increase in ASR. Finally, we find that the Recovering module is essential, as it preserves classification accuracy on clean data. Without the recovery phase, **GBND-RN** achieves a lower ASR than GBND by directly forgetting all backdoor neurons, effectively reducing their impact. However, since some of these neurons also contribute to clean node classification, its ACC drops significantly, further highlighting the importance of recovery.

4.4 Parameter Analysis (Q4)

We investigate the sensitivity of GBND on the Cora dataset under UGBA by varying its key parameters, α and t . The parameter α controls the balance between node feature preservation and structural consistency, while t adjusts the threshold for anomaly detection. As shown in Fig. 4, we vary α from 0.1 to 5.0 and t from 0.45 to 0.65 to assess their influence on the ASR and ACC. The results indicate that our model maintains stable performance across different parameter values, signifying its robustness against adversarial manipulations. Notably, we observe that setting $\alpha = 2.0$ and $t = 0.55$ yields the best

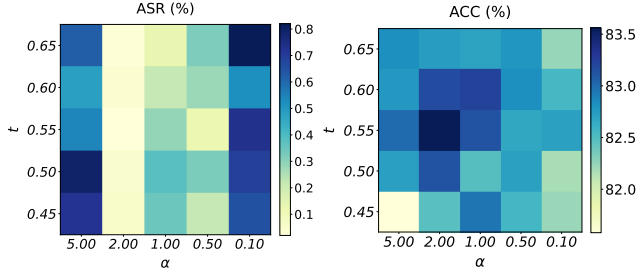


Figure 4: Experimental results for parameter α , and t .

performance, with the lowest ASR and highest ACC, suggesting this configuration optimally balances robustness and classification performance. This demonstrates the efficacy of our approach in eliminating backdoor attacks.

4.5 Defense on Different Trigger Pattern (Q5)

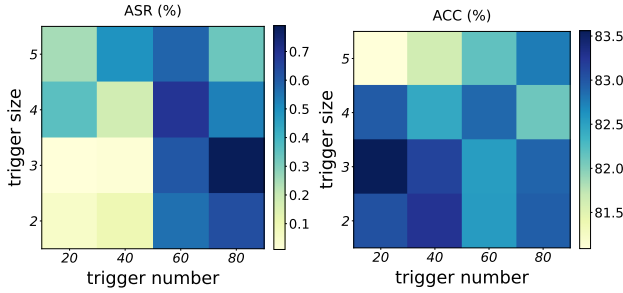


Figure 5: Experimental results for different trigger size and trigger number.

For defenders, the exact parameters of the triggers are typically unknown in advance. Therefore, we evaluated the robustness of our method under varying trigger sizes $\{2, 3, 4, 5\}$ and trigger numbers $\{20, 40, 60, 80\}$ for Cora, with other parameters kept constant. The results are presented in Fig. 5. We observe that neither the trigger size nor the trigger number significantly affects the defense effectiveness of our method. This robustness arises from the design of our approach, which targets the underlying trigger propagation pathways rather than focusing solely on the superficial properties of individual triggers.

Specifically, our neuron-level defense mechanism identifies and mitigates backdoor neurons that carry the malicious information across different trigger patterns, effectively neutralizing the attack regardless of the trigger's spatial or structural characteristics. This generalization capability allows our method to maintain high defensive performance across a wide range of trigger configurations, making it less sensitive to variations in trigger size and number.

5 Related Works

Graph Backdoor Attacks. Backdoor attacks on graphs have received extensive attention in recent years. Zhang et al. [32] propose a backdoor attack method specifically tailored to graph classification tasks, where the features of trigger nodes remain fixed. In the

same year, GTA [26] introduced a method for learning dynamic triggers and, for the first time, systematically applied backdoor attacks to node classification tasks. Subsequently, UGBA [2] addresses the challenge of conducting effective backdoor attacks under a limited attack budget by employing an adaptive trigger generator to craft subtle triggers that are difficult to detect. Additionally, DPGBA [33] further advances this line of work by generating in-distribution triggers for backdoor attacks on GNNs, utilizing out-of-distribution (OOD) detectors and adversarial learning to create subtle, undetectable triggers that enhance attack success rates while evading existing defenses. GRBA [3] further extended backdoor attacks to unsupervised representation learning tasks.

Graph Backdoor Defenses. Despite the progress in developing backdoor attacks, research on defenses against such attacks remains sparse, with most existing methods focusing on data-level processing techniques. For instance, the triggers generated by GTA exhibit low similarity to target nodes, allowing for their elimination through edge pruning based on similarity measures. Likewise, the triggers generated by GTA and UGBA exhibit deviations from normal data distributions, rendering them detectable via outlier detection. Additionally, RIGBD [34] addresses diverse trigger types by integrating random edge-dropping perturbations with robust model retraining. LoSplit [9] proposes a training-time backdoor defense framework that identifies target nodes by exploiting their abnormal early-stage loss dynamics during training. However, even this advanced strategy does not explicitly examine the GNN's internal inference pathways affected by the trigger. In contrast, our method targets these internal pathways, tracing and neutralizing the backdoor's effect within the model's learned representations, thereby disarming the backdoor trigger at its root.

6 Conclusion

In this paper, we introduce Graph Backdoor Neuron Defense (GBND), a novel and effective framework that eliminates backdoor neurons on Graph Neural Networks (GNNs) through neuron-level interventions. Specifically, GBND pinpoints and suppresses "backdoor neurons" without significantly impairing accuracy on clean nodes. To achieve this, we introduce a reverse engineering technique that learns minimal malicious triggers, enabling precise identification of backdoor neurons responsible for propagating harmful information. Furthermore, we achieve precise identification of these neurons by employing a reverse engineering technique that learns minimal malicious triggers, and we design an asymmetric unlearning and recovery process to effectively remove backdoors while preserving predictive capacity on clean nodes, ensuring stable and reliable model performance. In future work, we plan to integrate GBND with other robust learning strategies to further improve the robustness of GNN models in many real-world applications.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 92370111, 62272340, 62302333, 62422210, 62276187, 92570118, 62376088), Hebei Natural Science Foundation (No. F 2024202047) and Hebei Yanzhao Golden Platform Talent Gathering Programme Core Talent Project (Education Platform) (HJZD202509).

References

- [1] Shengwei An, Sheng-Yen Chou, Kaiyuan Zhang, Qiuling Xu, Guanhong Tao, Guangyu Shen, Siyuan Cheng, Shiqing Ma, Pin-Yu Chen, and Tsung-Yi Ho. 2024. Elijah: Eliminating backdoors injected in diffusion models via distribution shift. In *AAAI*. 10847–10855.
- [2] Enyan Dai, Minhua Lin, Xiang Zhang, and Suhang Wang. 2023. Unnoticeable backdoor attacks on graph neural networks. In *WWW*. 2263–2273.
- [3] Bingdao Feng, Di Jin, Xiaobao Wang, Fangyu Cheng, and Siqi Guo. 2024. Backdoor attacks on unsupervised graph representation learning. *Neural Networks* 180 (2024), 106668.
- [4] Bingdao Feng, Di Jin, Xiaobao Wang, Dongxiao He, Jingyi Cao, and Zhen Wang. 2025. Exploiting self-refining normal graph structures for robust defense against unsupervised adversarial attacks. In *IJCAI*. 2793–2801.
- [5] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NeurIPS*, Vol. 30.
- [6] Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. 2022. Datamodels: Predicting predictions from training data. In *ICML*.
- [7] Di Jin, Bingdao Feng, Siqi Guo, Xiaobao Wang, Jianguo Wei, and Zhen Wang. 2023. Local-global defense against unsupervised adversarial attacks on graphs. In *AAAI*, Vol. 37. 8105–8113.
- [8] Di Jin, Yujun Zhang, Bingdao Feng, Xiaobao Wang, Dongxiao He, and Zhen Wang. 2025. Backdoor attack on propagation-based rumor detectors. In *AAAI*, Vol. 39. 17680–17688.
- [9] Di Jin, Yuxiang Zhang, Bingdao Feng, Xiaobao Wang, Dongxiao He, and Zhen Wang. 2025. LoSplit: Loss-guided dynamic split for training-time defense against graph backdoor attacks. In *NeurIPS*.
- [10] Lingxin Jin, Xiangyu Wen, Wei Jiang, Jinyu Zhan, and Xingzhi Zhou. 2025. Trojan attacks and countermeasures on deep neural networks from life-cycle perspective: A review. *Comput. Surveys* (2025).
- [11] Alaa Khaddaj, Guillaume Leclerc, Aleksandar Makelov, Kristian Georgiev, Hadi Salman, Andrew Ilyas, and Aleksander Madry. 2023. Rethinking backdoor attacks. In *ICML*. 16216–16236.
- [12] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *ICLR*.
- [13] Runze Li, Di Jin, Xiaobao Wang, Dongxiao He, Bingdao Feng, and Zhen Wang. 2025. Single-node trigger backdoor attacks in graph-based recommendation systems. In *IJCAI*.
- [14] Yiming Li, Yong Jiang, Zhifeng Li, and Shu-Tao Xia. 2022. Backdoor learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 1 (2022), 5–22.
- [15] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. 2021. Neural attention distillation: Erasing backdoor triggers from deep neural networks. In *ICLR*.
- [16] Yige Li, Xixiang Lyu, Xingjun Ma, Nodens Koren, Lingjuan Lyu, Bo Li, and Yungang Jiang. 2023. Reconstructive neuron pruning for backdoor defense. In *ICML*. 19837–19854.
- [17] Junyu Lin, Lei Xu, Yingqi Liu, and Xiangyu Zhang. 2020. Composite backdoor attack for deep neural network by mixing existing benign features. In *CCS*. 113–131.
- [18] Yingqi Liu, Guangyu Shen, Guanhong Tao, Zhenting Wang, Shiqing Ma, and Xiangyu Zhang. 2022. Complex backdoor detection by symmetric feature differencing. In *CVPR*. 15003–15013.
- [19] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph attention networks. In *ICLR*.
- [20] Sebastian Vlačić, Theresia Conrad, Christian Tokarski-Schnelle, Mika Gustafsson, Uta Dahmen, Reinhard Guthke, and Stefan Schuster. 2018. ModuleDiscoverer: Identification of regulatory modules in protein-protein interaction networks. *Scientific reports* 8, 1 (2018), 433.
- [21] Binghui Wang, Jinyuan Jia, Xiaoyu Cao, and Neil Zhenqiang Gong. 2021. Certified robustness of graph neural networks against adversarial structural perturbation. In *SIGKDD*. 1645–1653.
- [22] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y Zhao. 2019. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE S&P*. IEEE, 707–723.
- [23] Xiaobao Wang, Ruoxiao Sun, Yujun Zhang, Bingdao Feng, Dongxiao He, Luzhi Wang, and Di Jin. 2025. Stealthy yet effective: Distribution-preserving backdoor attacks on graph classification. In *NeurIPS*.
- [24] Xiaobao Wang, Yujing Wang, Dongxiao He, Zhe Yu, Yawen Li, Longbiao Wang, Jianwu Dang, and Di Jin. 2025. Elevating knowledge-enhanced entity and relationship understanding for sarcasm detection. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [25] Dongxian Wu and Yisen Wang. 2021. Adversarial neuron pruning purifies backdoored deep models. *NeurIPS* 34 (2021), 16913–16925.
- [26] Zhaohan Xi, Ren Pang, Shouling Ji, and Ting Wang. 2021. Graph backdoor. In *USENIX Security*. 1523–1540.
- [27] Jing Xu and Stjepan Picek. 2023. Poster: Multi-target & multi-trigger backdoor attacks on graph neural networks. In *CCS*. 3570–3572.
- [28] Yachao Yang, Yanfeng Sun, Fujiao Ju, Shaofan Wang, Junbin Gao, and Baocai Yin. 2023. Multi-graph Fusion Graph Convolutional Networks with pseudo-label supervision. *Neural Networks* 158 (2023), 305–317.
- [29] Hao Yu, Ke Liang, Dayu Hu, Wenxuan Tu, Chuan Ma, Sihang Zhou, and Xinwang Liu. 2024. Chongqing GZOO: Black-box node injection attack on graph neural networks via zeroth-order optimization. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [30] Hao Yu, Chuan Ma, Meng Liu, Tianyu Du, Ming Ding, Tao Xiang, Shouling Ji, and Xinwang Liu. 2025. G²uardFL: Safeguarding federated learning against backdoor attacks via attributed client graph clustering. *IEEE Transactions on Information Forensics and Security* (2025). doi:10.1109/TIFS.2025.3639985
- [31] Xiang Zhang and Marinka Zitnik. 2020. GnnGuard: Defending graph neural networks against adversarial attacks. *NeurIPS* 33 (2020), 9263–9275.
- [32] Zaixi Zhang, Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong. 2021. Backdoor attacks to graph neural networks. In *SACMAT*. 15–26.
- [33] Zhiwei Zhang, Minhua Lin, Enyan Dai, and Suhang Wang. 2024. Rethinking graph backdoor attacks: A distribution-preserving perspective. In *SIGKDD*. 4386–4397.
- [34] Zhiwei Zhang, Minhua Lin, Junjie Xu, Zongyu Wu, Enyan Dai, and Suhang Wang. 2025. Robustness Inspired Graph Backdoor Defense. In *ICLR*.
- [35] Dingyuan Zhu, Ziwei Zhang, Peng Cui, and Wenwu Zhu. 2019. Robust graph convolutional networks against adversarial attacks. In *SIGKDD*. 1399–1407.
- [36] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. 2018. Adversarial attacks on neural networks for graph data. In *SIGKDD*. 2847–2856.

A Appendix

A.1 Overview of graph backdoor attack

Fig. 6 illustrates the general framework of a backdoor attack on graphs. In the training phase, the adversary injects triggers into the graph, modifying the features or structures of a subset of nodes, while simultaneously changing the labels of these target nodes to the adversary’s desired target label. This subtle manipulation creates a shortcut for the model, effectively associating the triggers with the target nodes and the target label.

During testing, this association remains intact, allowing the adversary to misclassify any node into the target label simply by adding a trigger, even if the node originally belonged to a different class. This occurs because the model has inadvertently learned to rely on the presence of the trigger as a shortcut for predicting the target label, bypassing the true structural or feature-based relationships within the graph.

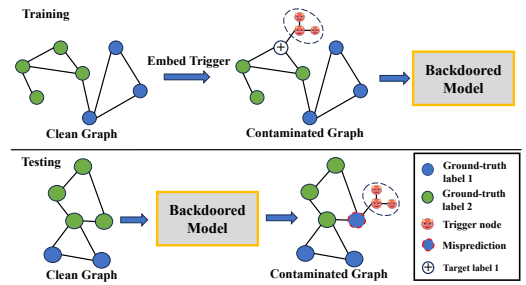


Figure 6: General framework of graph backdoor attack.

A.2 Proofs of Theorems

Proof of Theorem 3.1. Consider a graph G and a backdoored GCN model f_{θ} . This model predicts each normal node u as y_c . Moreover, a trigger node with an initial feature δ is embedded around node u , causing u to be misclassified as y_t .

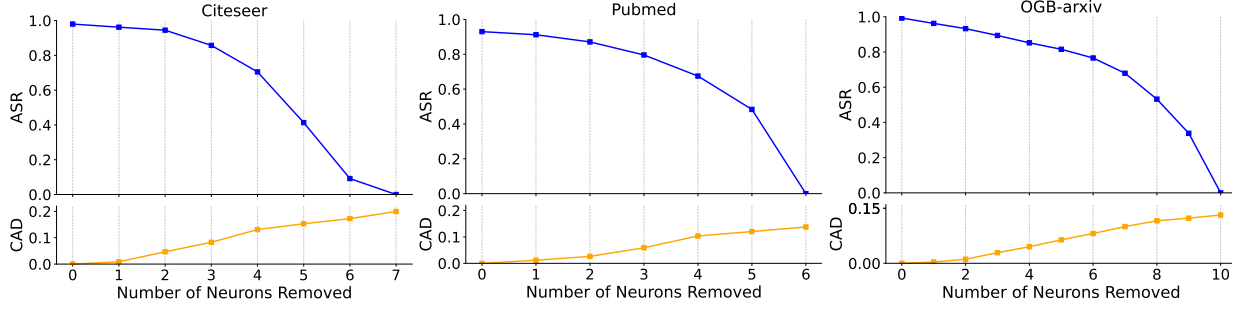


Figure 7: Impacts of neuron pruning on ASR (blue) and CAD (orange) for Citeseer, Pubmed, and OGB-arxiv.

Characterizing Minimal Backdoor Features in One-Layer GCNs. For a one-layer GCN, the embedding H_u for node u can be expressed as:

$$H_u = H_u^c + H_u^\delta, \quad (10)$$

where

$$H_u^c = \frac{1}{\sqrt{d_u}} X_u W + \sum_{v \in N(u)} \frac{1}{\sqrt{d_v d_u}} X_v W \quad (11)$$

is the normal feature aggregation, and

$$H_u^\delta = \frac{1}{\sqrt{d_u}} \delta W \quad (12)$$

is the backdoor feature contribution.

To ensure node u is classified as the target class y_t instead of the correct class y_c , we require:

$$(H_u)_t > (H_u)_c, \quad (13)$$

which, after substituting equations (11) and (12), becomes:

$$\frac{1}{\sqrt{d_u}} (\delta W)_t - \frac{1}{\sqrt{d_u}} (\delta W)_c > (H_u^c)_c - (H_u^c)_t. \quad (14)$$

Simplifying this, we get:

$$\delta(W_t - W_c) > \sqrt{d_u} ((H_u^c)_c - (H_u^c)_t), \quad (15)$$

where

$$D = (H_u^c)_c - (H_u^c)_t > 0. \quad (16)$$

Thus, the minimum backdoor feature satisfies:

$$\delta > \sqrt{d_u} \frac{D}{W_t - W_c}. \quad (17)$$

This completes the proof for the one-layer GCN case.

Characterizing Minimal Backdoor Features in Two-Layer GCNs. Consider a two-layer GCN with a single trigger node v_t and a target node u . The embedding of the target node u after the first layer is:

$$h_u^{(1)} = \sigma \left(h_u^c + \frac{1}{\sqrt{d_u}} \delta W \right), \quad (18)$$

where

$$h_u^c = \frac{1}{\sqrt{d_u}} x_u W + \sum_{v \in N(u) \setminus \{v_t\}} \frac{1}{\sqrt{d_v d_u}} x_v W \quad (19)$$

is the normal feature aggregation at the first layer.

Applying a first-order Taylor expansion:

$$h_u^{(1)} \approx h_u^c + \sigma' \left(h_u^c \right) \left(\frac{1}{\sqrt{d_u}} \delta W \right), \quad (20)$$

where the derivative σ' depends on the choice of the activation function, such as ReLU or Leaky ReLU. The error term for this approximation is given by:

$$E = \frac{1}{2} \sigma''(z) \left(\frac{1}{\sqrt{d_u}} \delta W \right)^2, \quad (21)$$

where z is a point between h_u^c and $h_u^c + \frac{1}{\sqrt{d_u}} \delta W$.

The trigger node's first-layer embedding is:

$$h_{v_t}^{(1)} = \sigma \left(\frac{1}{\sqrt{d_t}} \delta W + \frac{1}{\sqrt{d_t d_u}} x_u W \right), \quad (22)$$

which can be similarly approximated as:

$$h_{v_t}^{(1)} \approx \frac{1}{\sqrt{d_t}} \delta W + \frac{1}{\sqrt{d_t d_u}} x_u W + \sigma' \left(h_{v_t}^c \right) \left(\frac{1}{\sqrt{d_t d_u}} x_u W \right), \quad (23)$$

with the corresponding error term:

$$E_t = \frac{1}{2} \sigma''(z_t) \left(\frac{1}{\sqrt{d_t}} \delta W + \frac{1}{\sqrt{d_t d_u}} x_u W \right)^2. \quad (24)$$

Now, the second-layer embedding for node u is:

$$\begin{aligned} h_u^{(2)} &\approx h_u^c W^{(1)} + \sigma' \left(h_u^c \right) \left(\frac{1}{\sqrt{d_u}} \delta W \right) W^{(1)} \\ &\quad + \frac{1}{\sqrt{d_u}} \sigma' \left(h_{v_t}^c \right) \left(\frac{1}{\sqrt{d_t}} \delta W + \frac{1}{\sqrt{d_t d_u}} x_u W \right) W^{(1)}, \end{aligned} \quad (25)$$

with the total perturbation:

$$\begin{aligned} \delta_{\text{total}} &= \sigma' \left(h_u^c \right) \left(\frac{1}{\sqrt{d_u}} \delta W \right) W^{(1)} \\ &\quad + \sigma' \left(h_{v_t}^c \right) \left(\frac{1}{\sqrt{d_t}} \delta W + \frac{1}{\sqrt{d_t d_u}} x_u W \right) W^{(1)}. \end{aligned} \quad (26)$$

Therefore, a successful backdoor attack can be achieved as long as the induced perturbation δ_{total} satisfies the above condition.

A.3 Motivational Experiments for Neuron Pruning

We conduct additional experiments to further validate the effectiveness of neuron pruning in eliminating backdoor effects in the context of UGBA. In these experiments, we use 32-dimensional embeddings for Citeseer and Pubmed, and 64-dimensional embeddings for OGB-arxiv, consistent with the initial model settings. As shown in Figure 7, as the number of pruned neurons increases, the Attack Success Rate (ASR) (blue) drops sharply to 0, while the Clean Accuracy Drop (CAD) (orange) remains relatively moderate. This indicates that the neurons responsible for this rapid ASR reduction are likely backdoor neurons, as their removal effectively eliminates the attack.

A.4 Performance on Different GNNs

Table 4: Defense Performance on Different Surrogate Models (ASR and ACC in %).

Attacks	Model	Cora		Citeseer		PubMed	
		ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑
GTA	GAT	0.00	83.19	0.34	74.09	0.00	79.53
	GraphSage	1.98	80.33	0.68	72.27	0.22	76.85
UGBA	GAT	0.83	84.42	0.73	74.92	0.36	80.78
	GraphSage	1.30	81.86	1.69	73.43	0.77	77.04
DPGBA	GAT	2.93	82.06	0.26	73.82	0.24	79.50
	GraphSage	1.95	79.54	0.86	71.84	1.81	75.86

To demonstrate the transferability of our method, we evaluate the effectiveness of GBND on different GNN architectures. In addition to GCN, we also test our defense against GAT [19] and GraphSAGE [5] under various backdoor attack scenarios. The results are summarized in Tab. 4. We observe that GBND maintains strong defensive performance across different GNN models, consistently reducing the attack success rate (ASR) while preserving high clean accuracy (ACC). The fundamental reason for this robustness lies in the neuron-level defense mechanism of GBND, which targets the underlying trigger pathways rather than relying on specific graph structures or node feature distributions. Specifically, by directly identifying and pruning the neurons activated by backdoor triggers, GBND effectively disrupts the attack’s propagation, regardless of the message-passing mechanisms used in different GNN architectures. This neuron-centric approach ensures that the defensive effect is not limited to a specific model architecture, making it highly transferable across various GNN frameworks.